WebSocket-Native AI Coding Agent Architecture: A Deep Dive Synthesis

Executive Summary

This report synthesizes the internal architecture of Claude Code (decoded from its leaked npm source map), OpenAI's official Codex agent loop engineering deep-dive, the OpenAI Responses API WebSocket specification, community analysis from Chinese-language technical communities, and developer feedback from Reddit, Hacker News, and GitHub. The goal is to identify every "best part" across these tools and design a **novel, WebSocket-native, subagent-first, hook-governed AI coding agent** that is architecturally superior to all of them individually.

The central thesis: Claude Code's most impressive engineering is its **agent operating system** — the hook lifecycle, subagent orchestration, skill packaging, memory hierarchy, and prompt cache economics. Its biggest weakness is that every one of its sophisticated context management systems (3-tier compaction, full-history re-assembly, SYSTEMPROMPTDYNAMICBOUNDARY caching) exists solely to paper over **HTTP statelessness**. The moment you replace HTTP with WebSockets via the Responses API, those workarounds become unnecessary, and the architecture becomes dramatically cleaner.

Part 1: Claude Code Deconstruction

1.1 Source and Methodology

Claude Code v2.1.8 is distributed as an 860MB npm package. Its cli.js.map source map contains the full source of all 4,756 source files with original type annotations intact, enabling complete architectural reconstruction. The analysis below is derived from that reconstruction, cross-validated with Anthropic's official documentation and the community analysis in the attached files.[1]

1.2 The Agent Operating System Concept

Claude Code is best understood not as a chatbot with shell access, but as an **Agent Operating System** — a runtime that manages prompt assembly, tool governance, permission enforcement, agent lifecycle, skill packaging, plugin loading, and MCP integration as peer-level primitives. The core files that implement this OS are:[1]

- src/constants/prompts.ts — system prompt assembly
- src/tools/AgentTool/AgentTool.tsx + runAgent.ts — subagent runtime
- src/services/tools/toolExecution.ts — tool pipeline
- src/services/tools/toolHooks.ts — hook policy layer
- src/utils/plugins/loadPluginCommands.ts — plugin loading
- src/memdir/ — memory prompt hierarchy

The command surface exposed to users (/mcp, /memory, /permissions, /hooks, /plugin, /reload-plugins, /skills, /tasks, /plan, /review, /status, /model, /output-style, /agents, /sandbox-toggle) reveals the full breadth of the OS.[1]

1.3 The Prompt Assembly Architecture

The main system prompt is built by getSystemPrompt() as a layered assembly of sections, with a critical architectural detail: a SYSTEMPROMPTDYNAMICBOUNDARY marker that separates the **static cached prefix** (loaded once, reused every turn via Anthropic's prompt cache) from the **dynamic session-specific suffix** (rebuilt each turn).[1]

The static prefix sections assembled before the boundary are:

- getSimpleIntroSection() — identity and output style, URL handling, cyber-risk instructions
- getSimpleSystemSection() — runtime reality, permission mode, tool result format, prompt injection awareness, hooks runtime
- getSimpleDoingTasksSection() — coding standards: comments, docstrings, error handling, fallback, validation, future-proofing, compatibility
- getActionsSection() — destructive operations policy, blast-radius awareness for merges, lock files
- getUsingYourToolsSection() — tool usage grammar: FileRead equivalents to cat/head/tail/sed, FileEdit to sed/awk, Glob, Grep, Bash
- getSimpleToneAndStyleSection() — no emoji, file/line reference format, GitHub issue format
- getOutputEfficiencySection() — token budget awareness, brief responses, proactive behaviors

The dynamic suffix (rebuilt each turn) carries: session-specific guidance (AskUserQuestion enablement, feature gates, agent availability, slash skill commands), memory prompt (CLAUDE.md hierarchy), MCP server instructions, scratchpad function result clearing, summarize tool results config, numeric length anchors, and model/environment overrides.[1]

This static/dynamic separation is the foundational performance optimization — the expensive 4,756-file system prompt is tokenized once and cached. Every subsequent turn sends only the delta. **This is prompt cache economics as a first-class architectural constraint.**

1.4 The 8-Step Agentic Loop (Deconstructed)

Based on source map analysis and cross-validation with the ZenML/Codex architectural documentation, Claude Code's actual execution loop is:[2]

1. **Assemble full context** — system prompt (with cache boundary), CLAUDE.md memory hierarchy, full conversation history, attachments, MCP server instructions
2. **Call model API** — streaming async generator, Anthropic /v1/messages endpoint
3. **Parse response** — text blocks and tool_use blocks separated

4. **Permission gate** — `PreToolUse` hooks fire, `resolveHookPermissionDecision` returns allow/ask/deny
5. **Execute tools** — reads in parallel, writes serially; Bash speculative classifier check; `validateInput` (Zod schema); OTel analytics tracing
6. **Feed results back** — `PostToolUse` hooks fire; tool results appended to conversation
7. **Context management** — if context exceeds threshold, 3-tier compaction strategy triggers
8. **Loop termination check** — if no `tool_use` blocks in response, session complete[3][1]

The key insight from the attached analysis: **Steps 3–6 and 8 are entirely transport-agnostic**. They are pure harness logic that can run identically over WebSocket. Only Steps 1, 2, and 7 are architecturally coupled to HTTP, and only because HTTP is stateless — the server knows nothing between calls, so the client must carry everything.[3]

1.5 The Built-In Agent Roster

Claude Code ships six built-in agent specializations:[4][1]

Agent	Model	Tools	Role
General Purpose Agent	Sonnet (default)	Full tool set	Default task executor
Explore Agent	Haiku (fast)	Read-only (FileRead, Glob, Grep, Bash ls/git/grep/cat/head/tail)	Codebase analysis without modifying context
Plan Agent	Sonnet	Read + TodoWrite	Step-by-step implementation planning
Verification Agent	Sonnet	Bash (tests, linters, typecheck, curl)	Adversarial validator — returns VERDICT: PASS/FAIL/PARTIAL
Claude Code Guide Agent	Haiku	Read-only	Tool/feature guidance
Statusline Setup Agent	Haiku	Restricted	Terminal statusline configuration

The Explore Agent is architecturally important: it uses Haiku (cheaper, faster), is restricted to read-only tools, and returns results to the parent agent **without those results cluttering the main conversation context**. This is the key model for cost-efficient parallelism.[4][1]

The Verification Agent is equally significant — it uses a specific adversarial prompt ("try to break it") and generates structured verdicts across: verification avoidance, UI/CLI test coverage, test suites, linters, typechecks, frontend/backend endpoints, exit codes,

migration up/down, public API surfaces, and adversarial probes. This is institutionalized adversarial testing as a first-class agent primitive.[1]

1.6 Fork Path vs. Normal Path

The subagent spawning architecture has two distinct paths with different caching economics:[1]

Fork path (subagenttype: fork): Clones the parent's exact system prompt and context prefix (byte-identical). This enables **prompt cache inheritance** — the child agent's API request shares the same cached prefix as the parent, meaning there is no additional tokenization cost for the large system prompt. Fork agents are used for parallel independent tasks.[1]

Normal path (all other built-in and custom agents): Builds a fresh agent briefing. The agent definition provides description, prompt, model, runInBackground, name/teamName, isolation, cwd, and MCP server requirements. The isolation field determines whether the agent gets a worktree, remote execution, or in-process execution.[1]

1.7 Background vs. Foreground Agent Paths

Background agents run asynchronously with their own AbortController, generate a summary outputFile, and fire notifications. They use a summarization step to condense results before returning to the parent.[1]

Foreground agents block the main loop, show progress tracking, and integrate into the main conversation stream.[1]

This distinction matters for architecture: background agents are the right model for parallelizable work (file exploration, docs lookup, test runs) while foreground agents are right for sequential reasoning chains.

1.8 The Skills Primitive

Skills are Claude Code's most underappreciated architectural feature. A Skill is a first-class primitive — a Markdown file with YAML frontmatter that bundles:[1]

- A system prompt (workflow instructions)
- Allowed tools list
- Effort hints
- Model preferences
- Invocability flags (disablemodelinvocation)
- Slash command surface

Skills are loaded via CLAUDESKILLDIR, CLAUDEPLUGINROOT/CLAUDEPLUGINDATA environment variables. The DiscoverSkills system enables dynamic skill enumeration. Skills are effectively **declarative micro-agents** — they encode workflow packages that the orchestrator can load and invoke without code changes.[1]

The framework distinguishes three levels of extensibility:

1. **Skill** — workflow package (prompt + tools + metadata)
2. **Plugin** — CLI commands + skill commands + metadata + runtime constraints
3. **Hook** — lifecycle event handlers (shell commands or scripts)

1.9 The Hook System (Complete)

The hook system is the governance layer of the Agent OS. The full event roster:[5][6][7][1]

Event	When	Blocking?	Return Values
`SessionStart`	Session opens	No — stdout becomes Claude's context	System context injection
`PreToolUse`	Before any tool call	**Yes** — exit code 2 blocks	updatedInput, permissionBehavior (allow/ask/deny), preventContinuation, blockingError, stopReason
`PostToolUse`	After tool completes	No	`additionalContexts`, `updatedMCPToolOutput`
`PostToolUseFailure`	On tool error	No	Error context
`PermissionRequest`	Would show interactive permission dialog	No (skipped in `-p` mode)	Alternative to interactive gates
`Notification`	When agent sends notification	No	Alerting, logging
`Stop`	Agent finishes responding	No (can force continuation)	Quality gates, test runners
`StopFailure`	API error replaces Stop	No	Error handling

Event	When	Blocking?	Return Values
SubagentStart	Subagent spawns	No	Coordination logging
SubagentStop	Subagent finishes	No	Output validation
PreCompact	Before context compaction	No	Save context state
SessionEnd	Session terminates	No	Cleanup, memory extraction

The critical governance rule: **when multiple hooks match, the most restrictive answer wins** — a single deny from any hook cancels the tool call regardless of other results.[6]

SessionStart has a special capability: its stdout is injected directly into Claude's context, making it the mechanism for dynamic system prompt augmentation at session open without modifying the base prompt.[7]

1.10 The 7-Layer Memory Hierarchy

Claude Code's memory system is a cascading hierarchy of Markdown files assembled into the system prompt's Memory section:[8][9][3]

1. /etc/claude-code/CLAUDE.md — system-managed corporate rules (lowest priority)
2. ~/.claude/CLAUDE.md — personal cross-project preferences
3. ~/.claude/rules.md — granular personal rule files
4. Ancestor directory CLAUDE.md files — root-first traversal up to working directory
5. ./{project}/CLAUDE.md — project-level rules
6. ./{project}/.claude/rules.md — granular project rules
7. ./{project}/CLAUDE.local.md — gitignored local overrides (highest priority)[9][3]

An `@include` directive can pull in referenced documents up to 5 levels deep. Only the first 200 lines of any given file are auto-loaded; additional sections are on-demand.[10][8][3]

Critically, this is not "persistent memory" in the AI sense — it is developer-authored standing instruction text loaded at session start as a cached system prompt section. The model doesn't write to `CLAUDE.md` at runtime.[3]

The **actual agent-writable runtime memory** lives separately in `.claude/projects/{slug}/memory/MEMORY.md`, injected per turn as an attachment message (not baked into the system prompt). This is what the agent writes during a session to persist discoveries, user preferences, and past decisions across sessions.[3]

In architectural terms:

- `CLAUDE.md` → **Standing instructions** (equivalent to instructions field in OpenAI Responses API)
- `MEMORY.md` → **Runtime working memory** (equivalent to a tool result injected at context assembly time)

1.11 The 3-Tier Compaction System

Context management in Claude Code is a 3-tier strategy triggered when context approaches capacity:[3]

1. **Summarize tool results** — compresses individual tool output within the conversation
2. **Compact transcript** — calls `/responses/compact` equivalent via Anthropic's API to get a condensed window
3. **Context reset** — last-resort full reset for extremely long sessions

This entire system exists because HTTP is stateless. The server knows nothing between calls, so Claude Code must carry the full growing conversation history as payload every single turn. This is the core architectural tax of the HTTP transport.[3]

Part 2: OpenAI Codex Architecture

2.1 Official Engineering Anatomy (January 2026)

OpenAI published a detailed engineering deep-dive titled "Unrolling the Codex Agent Loop" in January 2026. Key revelations:[11][12][13]

Codex CLI, Codex Cloud, and VS Code extension all share the same "harness" — the core agent loop and execution logic. This is architecturally significant: the harness is transport-agnostic by design even if it currently uses HTTP.[2]

The Responses API is Codex's only inference interface. Endpoint routing:[12][2]

- ChatGPT login: `https://chatgpt.com/backend-api/codex/responses`
- API key: `https://api.openai.com/v1/responses`
- `--oss` flag (Ollama/LM Studio): `http://localhost:11434/v1/responses`
- Azure-hosted endpoints: fully supported[11][2]

Prompt assembly in Codex includes permissions, sandbox instructions, developer guidelines, aggregated user directives from `AGENTS.md`/`AGENTS.override.md`, optional skills blocks, and environment context.[14]

The append-only context strategy is the key cache optimization: Codex always appends new content to the end of the prompt, making the old prompt an exact prefix of the new one. This enables prompt cache hits — computation from previous inference calls is reused on the matching prefix. The result is linear rather than quadratic token computation as conversation grows.[15][12]

MCP is a cache hazard: MCP servers can change their tool list mid-session via `notifications/tools/list_changed`. Honoring this mid-conversation causes an expensive cache miss. Codex introduced a bug in its early MCP implementation by failing to enumerate tools in a consistent order.[14][12]

Automatic compaction: Codex uses a `/responses/compact` endpoint. The compact response returns a list including a special

type=compaction item with encrypted_content — an opaque blob that preserves the model's latent understanding of the conversation while reducing visible context. This now triggers automatically when auto_compact_limit is exceeded.[13][12]

2.2 Key Codex vs. Claude Code Architecture Differences

Dimension	Claude Code	OpenAI Codex CLI
Core philosophy	Agent OS (tools first, coding via tools)	Coding model (code expertise first, agent wrapper)
Provider	Anthropic /v1/messages (HTTP/SSE)	OpenAI Responses API (HTTP)
Subagents	6 built-in + unlimited custom	No native subagent system (community RFCs requesting this)[16]
Hooks	12 lifecycle events, blocking PreToolUse	User prompt hook added v0.116.0, limited[17]
Skills	First-class primitive (Markdown bundles)	AGENTS.md directive blocks
Memory	7-layer CLAUDE.md + agent-writable MEMORY.md	Single AGENTS.md file
Plugin system	Full CLI + runtime extension system	Limited, improving[17]
MCP support	Additive per-agent MCP servers	Single MCP configuration
Community	Massive (dominant in agentic workflows)	67K GitHub stars, 400+ contributors[17]
Developer rating	"Core Agent + Coding capability"[18]	"Core Model + Agent wrapper"[18]

Part 3: The OpenAI Responses API WebSocket Mode

3.1 Official Specification

The Responses API WebSocket mode is officially documented at developers.openai.com/api/docs/guides/websocket-mode. Key facts:[19][20]

Endpoint: wss://api.openai.com/v1/responses

Performance: Approximately 40% speedup for long-chain tasks with more than 20 tool calls. This comes from eliminating per-turn TCP/TLS handshakes, header overhead, and full-history re-transmission.[21][22]

Incremental input: On continuation turns, you send **only** new input items (functioncalloutput items + optional new user message) plus previous_response_id. The server caches the last response state on the active connection.[19][3]

Zero Data Retention compatibility: Both store=false and ZDR are fully supported.[21][19]

Connection limits:

- Duration: 60 minutes maximum per connection[21][19]
- Multiplexing: None — one response in-flight per connection. Use multiple connections for parallel sub-agent tasks.[19]

Reconnection: When websocketconnectionlimitreached is received, open a new WebSocket and continue by passing previous_response_id from the last completed turn on the old socket.[3]

Compaction on reconnect: If the conversation is very long, call POST /v1/responses/compact (one HTTP call) to get a compacted context window, then resume on the new socket with that compacted context.[3]

3.2 Event Protocol

The WebSocket event schema:[23][19][3]

Client sends (all via `response.create` messages):

```
{
  "type": "response.create",
  "model": "gpt-5.4",
  "store": false,
  "previous_response_id": "resp_...",
  "instructions": "You are a coding agent...",
  "input": [
    {"type": "function_call_output", "call_id": "...", "output": "..."},
    {"type": "message", "role": "user", "content": [...]}
  ],
  "tools": [...]
}
```

Server emits (streaming events):[23][19]

- `response.created` — carries `response_id`
- `response.output_text.delta` — streaming text chunks
- `response.output_item.done` — complete tool call (`type: function_call` with `name`, `arguments`, `call_id`)
- `response.done` — terminal event for the turn

3.3 The OpenAI Agents SDK Production Implementation

The OpenAI Agents Python SDK formalizes this architecture via the `responses_websocket_session` context manager:[24][25]

```
async with responses_websocket_session() as ws:
    result = await ws.run(starting_agent=agent, input=user_message)
```

Internally, this creates an `OpenAIResponsesWSModel` that:

- Keeps a shared WebSocket connection warm across multiple `Runner.run()` calls

- Preserves prefix-based model routing (`openai/gpt-4.1` syntax)
- Passes the same `RunConfig` to nested agent-as-tool runs, so they inherit the same connection[24][23]

`OpenAIResponsesWSModel` is defined as: *"Implementation of Model that uses the OpenAI Responses API over a websocket transport. The websocket transport currently sends `response.create` frames and always streams events."*[23]

Part 4: The WebSocket Architectural Inversion

4.1 What Changes and What Doesn't

The architectural mapping between Claude Code's harness loop and the WebSocket Responses API:[3]

Claude Code (HTTP)	WebSocket-Native Replica
Step 1: Assemble full history every turn	Step 1: **Eliminated** — send `previous_response_id` + new inputs only
Step 7: 3-tier local compaction subsystem	Step 7: Collapses to POST /v1/responses/compact when needed
Context grows client-side	Context lives server-side on the connection
Token cost scales with conversation length	Token cost is always just the new incremental input
CLAUDE.md re-injected every session start	CLAUDE.md injected once in `instructions` field — lives on the connection
MEMORY.md injected as attachment per turn	**Still required** — agent-writable memory is genuinely dynamic
Fork path shares cache prefix within agent	Each subagent gets its own WebSocket with own `previous_response_id` chain
Compaction triggers at 92% context capacity	Compaction triggers at 60-minute limit or on reconnect

Steps 3–6 and 8 are identical between Claude Code's harness and a WebSocket-native replica. The permission gate, tool execution, result feeding, and termination check are pure harness logic with no transport dependency.[3]

4.2 The Event Mapping

Claude Code event	Responses API WebSocket event
`tool_use` block in response	`response.output_item.done` with `type: function_call`
Append tool_result to messages list	Send function_call_output items in next response.create
Stream text delta	`response.output_text.delta`
End of turn (no tool_use)	response.done with empty tool calls

4.3 The Connection Lifecycle Handler (Replaces Entire Compaction Subsystem)

The only **genuinely new** architectural concern in a WebSocket-native harness is connection lifetime management. This replaces the entire multi-tier compaction system with approximately 10 lines of logic:[3]

```
async def reconnect_and_continue(previous_response_id: str) -> WebSock
    ws = create_connection("wss://api.openai.com/v1/responses", headers=...)
    ws.send(response.create(
        previous_response_id=previous_response_id,
        input=[new_continuation_input]
    ))
    return ws
```

When `websocketconnectionlimitreached` fires, call this function. Thread `previous_response_id` across socket boundaries. For extremely long sessions, call `POST /v1/responses/compact` once before reconnecting to compress the window.

Part 5: The Novel Architecture — Design Blueprint

5.1 Design Principles

The novel tool should be governed by four meta-principles drawn from the analysis:

1. **Transport > Provider** — the WebSocket event bus is the system's backbone, not the model API's format. Model providers are swappable adapters.
2. **Governance > Speed** — the hook and permission system is not optional scaffolding; it is the architectural guarantee that makes aggressive automation safe.
3. **Specialization > Generality** — every subagent has a narrow role, narrow tools, and a narrow model. Generalist agents produce worse results and cost more.
4. **Memory is Operational** — the system stores decisions, patterns, and preferences, not conversation logs. Memory that doesn't improve future execution is waste.

5.2 The Three-Plane Architecture

The proposed architecture separates concerns into three planes:

Control Plane — owns sessions, turns, event routing, approval state, agent graph, auth, transcript, quotas, and WebSocket lifecycle. This is the `session_server` daemon that runs as a background process.

Reasoning Plane — owns planners, workers, reviewers, synthesis, memory extraction, and docs verification. These are the model-backed agents. They consume events from the Control Plane and produce tool call requests.

Execution Plane — owns shell, file operations, browser automation, test runners, patch application, MCP connectors, and sandboxes. This never calls a model. It only executes what the Reasoning Plane requests, subject to Control Plane policy.

This separation enables: multiple simultaneous frontends (CLI, IDE extension, web console), remote workspaces, centralized sandboxing,

and full event audit logs.

5.3 The Session Model

Every session carries durable, resumable state:[19][3][1]

```
session_id
├── thread_id
├── agent_graph
│   ├── coordinator (WebSocket connection + previous_response_id chain)
│   └── worker_pool[]
│       ├── explorer (own WS connection, read-only tools)
│       ├── implementer (own WS connection, write tools)
│       └── reviewer (own WS connection, bash/test tools)
├── approval_profile (default | accept-edits | full | plan-only)
├── memory_state
│   ├── project_instructions (CLAUDE.md hierarchy → instructions field)
│   ├── runtime_memory (MEMORY.md → per-turn attachment)
│   └── session_compacted_context
├── hook_state
├── mcp_registry
└── event_log (transcript, tool calls, approvals, timing)
```

5.4 The WebSocket Event Bus

The internal event schema (provider-agnostic, typed):[6][19][3]

```
// Control events
SessionStarted | ThreadResumed | TurnStarted | TurnCompleted | Ses

// Model events (from WebSocket)
ModelDelta | ModelTurnDone

// Tool lifecycle
ToolRequested | ApprovalRequired | ApprovalResolved | ToolStarted
ToolStdoutDelta | ToolCompleted | ToolFailed

// Hook lifecycle
```

```
HookFired | HookResult | HookBlocked | HookContinued

// Agent graph
SubagentSpawned | SubagentTask | SubagentResult | SubagentComple

// Memory
MemoryRead | MemoryWritten | MemoryCompacted

// Policy
PolicyViolation | PermissionEscalation | SandboxBreach
```

5.5 The Built-In Agent Specializations

Drawing from Claude Code's proven roster and extending it with OpenAI Codex patterns:[18][16][4][1]

Agent	Model Tier	Tools	Purpose
Coordinator	Strong planner (slow)	Agent spawn, synthesis	Plans, delegates, resolves conflicts
Explorer	Fast/cheap	Read-only: Glob, Grep, FileRead, git, find	Maps codebase, no write access
Implementer	Mid-tier coder	Read + Write + Edit	Writes targeted patches
Reviewer	Mid-tier	Read + Bash (no write)	Code review, security, correctness
Verifier	Mid-tier	Bash (tests, linter, typecheck)	PASS/FAIL/PARTIAL verdicts
Docs Researcher	Fast	Web fetch, FileRead	Verifies APIs against official docs
Browser Debugger	Mid-tier	Browser automation	Reproduces UI bugs
Test Analyst	Fast	Bash (test runner), FileRead	Isolates flaky tests
Memory Extractor	Fast	FileWrite (MEMORY.md only)	Extracts operational facts post-turn

Each agent is declared as a Markdown file with YAML frontmatter (following Claude Code's pattern):[4]

```
---
name: verifier
description: Adversarial validator that runs tests and returns VERDICT
```

```
model: mid-tier
tools: [bash]
permission_mode: accept-edits
max_depth: 0
hooks: [post-verify-notification]
---
```

5.6 The Hook Lifecycle (Extended)

Building on Claude Code's 12 hooks and adding hooks motivated by Codex patterns:[5][7][14][6]

Hook	Phase	Blocking?	Key Uses
SessionStart	Session open	No (context injection)	Dynamic context augmentation
PromptPreflight	Before model call	Yes	Prompt scanning, injection detection
PreToolUse	Before tool execution	**Yes**	Security gates, file protection, risk classification
PostToolUse	After tool completes	No	Auto-format, lint, logging
PostToolUseFailure	On tool error	No	Error telemetry
PermissionRequest	Interactive permission	No	Graduated approval
PatchValidation	After write/edit tool	No	Diff review, test trigger
Stop	Agent finishes	No (can force continue)	Quality gates
StopFailure	API error	No	Error handling
SubagentStart	Subagent spawns	No	Graph tracking
SubagentStop	Subagent finishes	No	Output validation
PreCompact	Before compaction	No	Save context state
MemoryEx	After stop	No	Operational memory

Hook	Phase	Blocking?	Key Uses
traction			extraction
RepoConventionCheck	After writes	No	Style guide enforcement
SessionEnd	Session terminates	No	Cleanup, persistence

Governance rule inherited from Claude Code: most restrictive wins when multiple hooks match.[6]

5.7 The Graduated Approval System

Building on Claude Code's permission modes and Codex's guardian pattern, approvals should be **policy state**, not popup spam:[1]

Level	Auto-Approved	Requires Confirmation
Read-only	All file reads, git log/status/diff	Nothing
Accept-edits	FileEdit, FileWrite within approved roots	New root additions
Default	Safe shell (git, npm test, etc.)	Network calls, system modifications
Full	All tools	Destructive commands
Plan-only	Nothing	Everything — planning pass only

Risk classification for Bash commands should use a speculative classifier (Claude Code does this) that categorizes commands as: safe/read-only, potentially-modifying, destructive, network-calling, secrets-accessing.[1]

Subagent approval inheritance: child agents inherit parent's approval profile but may be further narrowed, never widened.

Provenance metadata (who spawned, for what purpose, with what constraints) travels with every approval decision.[4][1]

5.8 The Memory Architecture

Three tiers, each with a distinct semantic role:

Tier 1 — Standing Instructions (CLAUDE.md hierarchy): Developer-authored, version-controlled, injected once into the instructions field of the initial response.create. Cascading scope by directory depth. Never written by the agent at runtime.[9][3]

Tier 2 — Runtime Working Memory (MEMORY.md): Agent-writable. Injected as an attachment message on every turn continuation. Contains operational discoveries: repo conventions, naming patterns, test commands that actually work, approved architectural choices, known failure signatures. Hook: MemoryExtraction after every Stop event — a fast, cheap model call extracts operationally significant facts and appends them.[26][8][3]

Tier 3 — Session Compacted Context: After the 60-minute connection limit is reached, the POST /v1/responses/compact response's encrypted_content is persisted as a resumable session token. On reconnect, this token is passed as context — the model's latent understanding of the conversation is preserved without re-reading the full transcript.[12][19][3]

Memory discipline rule: Store operational facts (commands, patterns, preferences, decisions), not conversation summaries. The 62% memory failure rate in Claude Code deployments comes from MEMORY.md containing outdated information that overrides current project rules.

5.9 The MCP Integration Strategy

MCP is the universal tool connector for external systems. Key architecture decisions motivated by the Codex cache miss problem:[27][28][14][12][1]

1. **Lock the tool list at session start** — enumerate all MCP tools once, validate schema, freeze for the session. Do not honor mid-

session notifications/tools/list_changed unless the user explicitly requests a tool refresh (which flushes and rebuilds the context).
2. **Sort tools deterministically** — alphabetical order within categories. Cache hits require byte-identical tool lists.[12]
3. **Per-agent MCP scoping** — each subagent type has a declared MCP allowlist. The Explorer agent cannot see write-capable MCP servers. This is directly from Claude Code's agentspecific MCP servers additive pattern.[1]
4. **Frontmatter-declared MCP** — agent definition files can declare required MCP servers in YAML frontmatter. The runtime initializes those servers before spawning the agent and cleans them up after.[1]

5.10 The Skill/Plugin Architecture

Three extension tiers, each with clear governance:

Skills (workflow packages): Markdown files with YAML frontmatter. Declare prompt, tools, model, effort, invocability. Loaded from SKILLDIR environment variable. First-class primitive — the orchestrator can discover and invoke skills without code changes. Equivalent to Claude Code's SkillTool.[1]

Plugins (capability bundles): Contain skills, hook scripts, MCP server configurations, and CLI commands. Deployed as a directory bundle. Plugins cannot execute arbitrary code in the core runtime unless running in an isolated subprocess. Analogous to Claude Code's loadPluginCommands.ts.[1]

MCP connectors (external tool bridges): Connect to external systems via the Model Context Protocol. Per-agent scoping. Tool list frozen at session start.[27][1]

5.11 Prompt Cache Economics (First-Class Constraint)

Both Claude Code and Codex treat prompt cache performance as a first-class architectural constraint. The rules:[12][1]

1. **Static content first, dynamic content last** — system prompt, standing instructions, tool definitions, static memory. Dynamic

content (current turn, tool results) always appended to the end.[15][12]
2. **Deterministic tool enumeration** — same order every time. Alphabetical + stable category ordering.[12]
3. **Freeze MCP tool lists** — mid-session tool list changes bust the entire cache.[14][12]
4. **Fork path for subagents** — fork subagents get a byte-identical context prefix to the parent, enabling cache inheritance. Costs only the fork delta, not a full context re-build.[1]
5. **Model constancy** — switching models mid-session busts the cache. The harness should warn before model switches.[14]
6. **WebSocket advantage** — the `previous_response_id` mechanism means the server maintains the cache state. The client doesn't need to re-send the static prefix at all — it only sends new incremental inputs.[19][3]

5.12 The Killer UX Moves

Beyond architecture, the product differentiation comes from experience design:

The one-agent illusion: Surface as a single conversational agent. Internally fan out to a specialized subagent network. The user sees one coherent assistant; the system runs a coordinated team.[29][18]

Live agent graph view: Optional debug/power-user panel showing which agents are active, what tools they're running, and their current task status. Every professional developer asked for "more transparency" in Reddit/HN threads.[30][18]

Typed child-task cards: Subagent tasks displayed as cards with type (explore | implement | verify | research), status, and estimated completion. Not raw logs.

Approval requests labeled by agent source: "Explorer wants to run `grep -r 'TODO' .`" not just "Claude wants to run a command."

Reversible patch stack: Every file modification tracked as a named patch. `/undo` reverts the last N patches. `/checkpoint` names a state. Sessions can be rewound.

Diff-to-decision provenance: Every code change linked to the tool call that created it, the agent that requested it, and the reasoning trace that motivated it.

MEMORY.md as a living document you can edit: The agent reads it, you can read it, you can edit it. No opaque vector store. Full transparency and human override.

Part 6: Developer Feedback Synthesis

6.1 Claude Code Strengths (Community Consensus)

- **Tool ecosystem** is the #1 cited advantage: "Claude Code functions as: (Core Agent) + Coding Capability. Its foundation lies in the Agent, which executes tasks by utilizing a variety of tools" [18]
- **Large codebase handling** — 1M token context enables whole-repo analysis that other tools fail at[29]
- **Multi-agent parallel work** — up to 16+ simultaneous agents reported in production[29]
- **Output quality** — "Claude's output tends to be cleaner and needs less cleanup"[30]
- **Memory and continuity** — CLAUDE.md hierarchy praised as "shockingly effective" in long-running projects[9]

6.2 Claude Code Weaknesses (Community Consensus)

- **HTTP overhead** in long agentic sessions is measurable and frustrating (multiple Reddit threads)
- **MEMORY.md conflicts** cause 62% of reported memory failures[8]
- **No built-in branch isolation** for parallel subagent work (worktree support is partial)
- **Tool permission dialogs** become fatiguing in automation contexts
- **No native WebSocket support** — requires proxies for OpenAI-compatible endpoints

6.3 Codex Strengths (Community Consensus)

- **Model quality on pure coding tasks**: "Codex operates as (Core Model) Agent — begins with an exceptionally deep grasp of code"[18]
- **Cloud-native parallel execution** — multiple tasks running simultaneously in sandboxed containers
- **Responses API native** — designed from the start for the modern inference API
- **67K GitHub stars, growing fast**[17]

6.4 Codex Weaknesses (Community Consensus)

- **No native subagent orchestration** — the GitHub community has an open RFC explicitly requesting this: "Pipelines with specialized roles (planner → coder → tester → reviewer) dramatically reduce hallucinations"[16]
- **No built-in prompt engineering layer** — teams must reinvent fragile prompt glue[16]
- **Hook system immature** — v0.116.0 adds a user prompt hook but the system is far less developed than Claude Code's[17]
- **Single AGENTS.md** vs. Claude Code's 7-layer memory hierarchy
- **Shallow on complex tasks** vs. deep on straightforward ones[31]

6.5 The Gap This Architecture Fills

The community feedback defines a clear gap: developers want **Claude Code's agent OS depth** (subagents, hooks, skills, memory hierarchy) running on **OpenAI's Responses API with WebSocket transport** (performance, clean API design, provider flexibility) with **neither system's weaknesses**.

The novel architecture proposed here is exactly that synthesis.

Part 7: Implementation Roadmap

7.1 Phase 1 — Core Loop (MVP)

- WebSocket session manager with `previous_response_id` chaining and 60-minute reconnection
- Claude Code 8-step harness loop (Steps 3–8) running over WebSocket transport
- Core tools: FileRead, FileEdit, FileWrite, Bash, Glob, Grep, TodoWrite
- `instructions`-field memory injection (CLAUDE.md hierarchy equivalent)
- MEMORY.md per-turn attachment injection
- PreToolUse / PostToolUse / Stop hook lifecycle
- CLI frontend with streaming delta rendering

7.2 Phase 2 — Agent Specialization

- Coordinator + Explorer + Implementer + Verifier agent definitions
- Fork path with cache prefix inheritance
- Background/foreground agent lifecycle (AbortController, summarization, outputFile)
- Per-agent tool restrictions and MCP scoping
- Graduated approval system (5 levels)
- Subagent-aware event bus (SubagentStart/Stop events)

7.3 Phase 3 — Memory and Skills

- Full 7-layer memory hierarchy with `@include` support
- Agent-writable MEMORY.md with `MemoryExtraction` hook
- Session compaction at 60-minute boundary (POST /v1/responses/compact)
- Skill primitive (Markdown + YAML frontmatter)
- Plugin bundle loader
- `/skills`, `/agents`, `/memory`, `/hooks` slash commands

7.4 Phase 4 — Observability and UX

- Live agent graph view (terminal TUI + web console)
- Typed task cards per subagent
- Reversible patch stack with `/undo`, `/checkpoint`
- Diff-to-decision provenance linking

- MCP tool registry with deterministic enumeration and session-locked lists
- IDE extension client (VS Code, JetBrains)

7.5 Phase 5 — Production Hardening

- Multi-provider adapter layer (OpenAI, Anthropic, Bedrock, Vertex)
- Remote execution substrate (Execution Plane as networked daemon)
- Signed/sandboxed plugin bundles
- Enterprise permission profiles and audit logging
- Cross-session memory compaction and knowledge graph

Part 8: Architecture Summary Matrix

Component	Source of Inspiration	Implementation
WebSocket event bus	OpenAI Responses API WebSocket mode[19]	`wss://api.openai.com/v1/responses`
previous_response_id chaining	OpenAI official spec[19]	Eliminates full-history re-send
60-min reconnection handler	OpenAI connection limits[19]	Replaces 3-tier compaction
/v1/responses/compact	Codex auto-compaction[12]	Single HTTP call on reconnect
8-step harness loop	Claude Code source map[1]	Steps 3–8 fully preserved
SYSTEMPROMPTDYNAMICBOUNDARY	Claude Code prompts.ts[1]	→ instructions field in WebSocket
7-layer memory hierarchy	Claude Code memdir[3][9]	Fully preserved
MEMORY.md per-turn injection	Claude Code attachment system[3]	Per-turn input item
6 built-in agents	Claude Code builtInAgents.ts[1][4]	Expanded to 9
Fork path cache inheritance	Claude Code AgentTool[1]	Per-agent WebSocket connection
12+ lifecycle hooks	Claude Code toolHooks.ts[6][5]	Extended to 15
Skill primitive	Claude Code SkillTool[1]	Markdown + YAML frontmatter

Component	Source of Inspiration	Implementation
Plugin system	Claude Code plugins[1]	Directory bundle + governance
MCP per-agent scoping	Claude Code additive MCP[1]	YAML frontmatter declaration
Prompt cache economics	Codex engineering deep-dive[12]	Static-first, deterministic tools
responses_websocket_session	OpenAI Agents SDK[24]	Shared WS session across agent runs
Graduated approvals	Claude Code permission modes[1]	5-level policy state
Three-plane separation	Codex harness/exec separation[2]	Control/Reasoning/Execution planes

---

Conclusion

The "monster" coding agent is not a choice between Claude Code's architecture and OpenAI's protocols. It is both, with their weaknesses removed. Claude Code has proven that **the agent OS model — hooks, skills, specialized subagents, and a layered memory hierarchy — produces qualitatively better results** than any simpler approach. OpenAI has proven that **the Responses API with WebSocket transport eliminates the stateless HTTP tax** that forces Claude Code to build elaborate compaction and context management systems.

The proposed architecture keeps every component of Claude Code's agent OS that earns its complexity, replaces every component that exists only to paper over HTTP statelessness with the WebSocket-native equivalent, and adds the observability, provenance, and reversibility layers that the developer community consistently identifies as missing from both tools. The result is architecturally superior on every measurable dimension: latency (40% WebSocket

speedup), context cost (incremental input only), agent sophistication (Claude Code's full OS), and developer experience (live graph view, patch stack, typed task cards).[21][19][1]

The key insight to anchor the build: **the 3-tier compaction system is the HTTP problem. Remove HTTP, the problem largely disappears.**[3]

References

1. claude-code-deep-dive-xelatex.pdf - Claude Code Claude Code Xiao Tan X x.comtvytlx Xiao Tan AI March 31, 2026 Contents 1 Claude Code 4 1...
2. OpenAI Codex CLI Architecture and Agent Loop Design - ZenML - OpenAI's Codex CLI is a cross-platform software agent that executes reliable code changes on local m...
3. openai-responses-api-websockets-mode-codex-compati.md - img srchttpsr2cdn.perplexity.aipplx-full-logo-primary-dark402x.png styleheight64pxmargin-right32px
4. Create custom subagents - Claude Code Docs - Create and use specialized AI subagents in Claude Code for task-specific workflows and improved cont...
5. Claude Code Hooks Reference: All 12 Events [2026] | Pixelmojo - Official-style reference for all 12 Claude Code hook events. PreToolUse, PostToolUse examples, CI/CD...
6. Automate workflows with hooks - Claude Code Docs - Run shell commands automatically when Claude Code edits files, finishes tasks, or needs input. Forma...
7. Claude Code Hooks: Automate Your AI Coding Workflow - PreToolUse and PostToolUse handle the before and after of tool execution. PostToolUseFailure fires o...
8. The CLAUDE.md Memory System - Deep Dive - SFEIR Institute - Claude Code implements a three-level hierarchy of memory files. Each level has a different scope and...
9. Claude Code's Memory: Working with AI in Large Codebases - Claude Code's memory system is deceptively simple on the surface — it's just Markdown files. But ben...
10. Claude Code's Memory System: The Full Guide (Most ... - YouTube - This video explores the advanced memory system of "claude

code", which operates with six layers of d...
11. OpenAI deep dive: “Unrolling the Codex agent loop” (how Codex actually builds prompts, calls Responses API, caches, and compacts context) - OpenAI deep dive: “Unrolling the Codex agent loop” (how Codex actually builds prompts, calls Respons...
12. Unrolling the Codex agent loop - By Michael Bolin, Member of the Technical Staff
13. Unrolling the Codex agent loop - Hacker News - This is a big deal and very useful for anyone wanting to learn how coding agents work, especially co...
14. OpenAI deep dive: “Unrolling the Codex agent loop” (how ... - Reddit - Prompt caching depends on exact prefix matches, so tool order and mid-run tool list changes (MCP) ca...
15. How AI coding agents work - by Bhavishya Pandit - A complete guide covering how AI coding agents work, including system design, execution flow, and pe...
16. RFC: Make Codex CLI state-of-the-art — built-in prompt ... - Context I’m an everyday GPT-5 user for professional development. The model is excellent, but the Cod...
17. OpenAI Codex CLI ships v0.116.0 with enterprise features - OpenAI Codex CLI hits 67K GitHub stars with v0.116.0. Here's what the terminal-native coding agent o...
18. My one-day deep dive on Codex vs. Claude Code vs. Cursor - My one-day deep dive on Codex vs. Claude Code vs. Cursor
19. WebSocket Mode | OpenAI API - Learn how to use Responses API WebSocket mode with response.create and previous_response_id.
20. Compaction and creating new... - Learn how to use Responses API WebSocket mode with response.create and previous_response_id.
21. OpenAI Adds WebSocket Support to the Responses API ... - KuCoin - OpenAI has added WebSocket support to its Responses API, improving long-chain task execution by 40%....
22. OpenAI adds WebSocket support to the Responses API ... - PANews reported on February 25th that, according to OpenAI developer documentation, OpenAI has intro...
23. OpenAI Responses model - OpenAI Agents SDK
24. Responses WebSocket Session - OpenAI Agents SDK - Create a shared OpenAI Responses websocket session for multiple

Runner calls. The helper returns a s...

25. How To Correct Implement WebSockets · Issue #2558 - GitHub - responses_websocket_session() already creates a websocket-enabled OpenAI provider internally, so if ...
26. The Architecture of Persistent Memory for Claude Code - This is baked into how Claude Code works. Whatever is in that file becomes part of Claude's initial ...
27. How AI Agents Use MCP for Enterprise Systems 2026 - AgileSoftLabs - The Model Context Protocol (MCP) is rapidly becoming the universal standard for AI agent integration...
28. Building effective AI agents with Model Context Protocol (MCP) - Learn how Model Context Protocol (MCP) enhances agentic AI in OpenShift AI, enabling models to call ...
29. Cursor vs Windsurf vs Claude Code: Best AI Coding Tool in 2026 ... - Most developers: Start with Cursor if you want a polished IDE experience. Switch to Windsurf if cost...
30. Claude vs Codex vs Cursor — what would you pick for serious side ... - Codex is solid too especially for complex tasks but Claude's output tends to be cleaner and needs le...
31. We Tested 15 AI Coding Agents (2026). Only 3 Changed How We ... - Codex is fast but shallow compared to Claude. Developers on HN consistently report that Codex handle...